

INFORME DE EVALUACIÓN Y SUSTENTACIÓN DE LA UNIDAD I

Sistema Distribuido Base Orientado a Producción (Sesiones 01 a 05)

Proyecto Sello: ChaskiPC Hardware E-Commerce

Estudiante: Eliceo Parillo Mostajo | **Equipo:** Equipo 01 - ChaskiByte Systems

Sección: 5to Ciclo - Grupo Único | **Docente:** Ing. Abel Ángel Sullón Macalupu

Proyecto Sello: ChaskiPC Hardware E-Commerce (Sistema Distribuido para la Venta de Computadoras y Componentes)

Repositorio GitHub: <https://github.com/jaisesasaki24-cloud/ChaskiByte-Systems>

Topics: campus-juliaca, semestre-2026-2, linea-software, tipo-ps, dist, seccion-g1, grupo-01-chaskipc

1. Resumen de Arquitectura del Sistema Distribuido Base

El ecosistema distribuido de ChaskiPC integra un punto único de acceso (Gateway en puerto 18080 DEV / 28080 PROD), un registro de servicios Eureka (8761 DEV / 28761 PROD), configuración centralizada por ambiente (8888 DEV / 28888 PROD), microservicios de catálogo y órdenes con ejecución concurrente y base de datos relacional en PostgreSQL (puerto 5433).

Componente	Rol en el Sistema	Puerto DEV (Local)	Puerto PROD (Docker)	Estado
pagatu-config	Configuración Centralizada	8888	28888	Operativo
pagatu-eureka	Service Registry & Discovery	8761	28761	Operativo
pagatu-gateway	Punto Único de Acceso & Load Balancer	18080	28080	Operativo
pc-catalogo-ms	Catálogo de Hardware (CPUs, GPUs, RAM)	8081	Red interna (8080)	Operativo
pc-orden-ms (Inst. 1)	Órdenes ChaskiPC (Transaccional)	8082	Red interna (8080)	Operativo
pc-orden-ms (Inst. 2)	Réplica Concurrente de Órdenes	8083	Red interna (8080)	Operativo
PostgreSQL (Docker)	Persistencia Relacional orden_db	5433	Red interna (5432)	Operativo
Prometheus	Métricas con Eureka Service Discovery	19090	29090	Operativo
Grafana	Dashboard Visual de Salud y JVM	13000	23000	Operativo

2. Balotario de Sustentación Teórico-Práctica (S1 a S4)

1. ¿Por qué tu microservicio no debería depender de un puerto fijo asignado a mano, y cómo verificaste que corre con múltiples instancias en paralelo?

Depender de un puerto fijo impide el escalado horizontal. Al usar instance-id: `${spring.application.name}:${server.port}` con Eureka, el microservicio se anuncia dinámicamente. Se verificó

arrancando una instancia en 8082 y otra en 8083; ambas se muestran vivas simultáneamente en Eureka (UP (2)) y atienden peticiones concurrentes compartiendo PostgreSQL.

2. ¿Qué diferencia hay entre una propiedad fija en el código y una leída desde tu Config Server, y por qué esa diferencia importa entre DEV y PROD?

Una propiedad fija exige recompilar el JAR ante cualquier cambio. Con Config Server, las configuraciones residen en config-repo: en DEV se usa la BD local con logs DEBUG y DDL update; en PROD se inyectan credenciales seguras y DDL validate, cumpliendo con el Factor III de los 12 Factores sin tocar el código.

3. Si detienes una instancia de tu servicio, ¿cómo se entera tu registro de que ya no está disponible, y por qué no es instantáneo?

El microservicio envía un heartbeat cada 30s. Si se apaga, Eureka espera la ventana de expiración del arrendamiento (90s) antes de desalojarla del catálogo. Esto previene que caídas momentáneas de red expulsen servicios sanos de forma prematura.

4. ¿Por qué la dirección lb:// no es una dirección real, y qué componente la resuelve?

lb:// es un esquema lógico de Spring Cloud LoadBalancer. No es una IP ni un DNS. El LoadBalancer consulta a Eureka para obtener la lista física de instancias vivas y enruta la petición hacia una de ellas.

5. ¿Qué algoritmo de balanceo usa tu Gateway por defecto, y por qué es suficiente para instancias sin estado?

Usa Round Robin. Es suficiente porque las instancias son idénticas y stateless; todo el estado vive en la base de datos PostgreSQL compartida, garantizando equidad de carga sin necesidad de afinidad de sesión.

3. Evidencias Técnicas del Sistema en Vivo

3.1 Dashboard de Eureka con Múltiples Instancias y Gateway Operativo

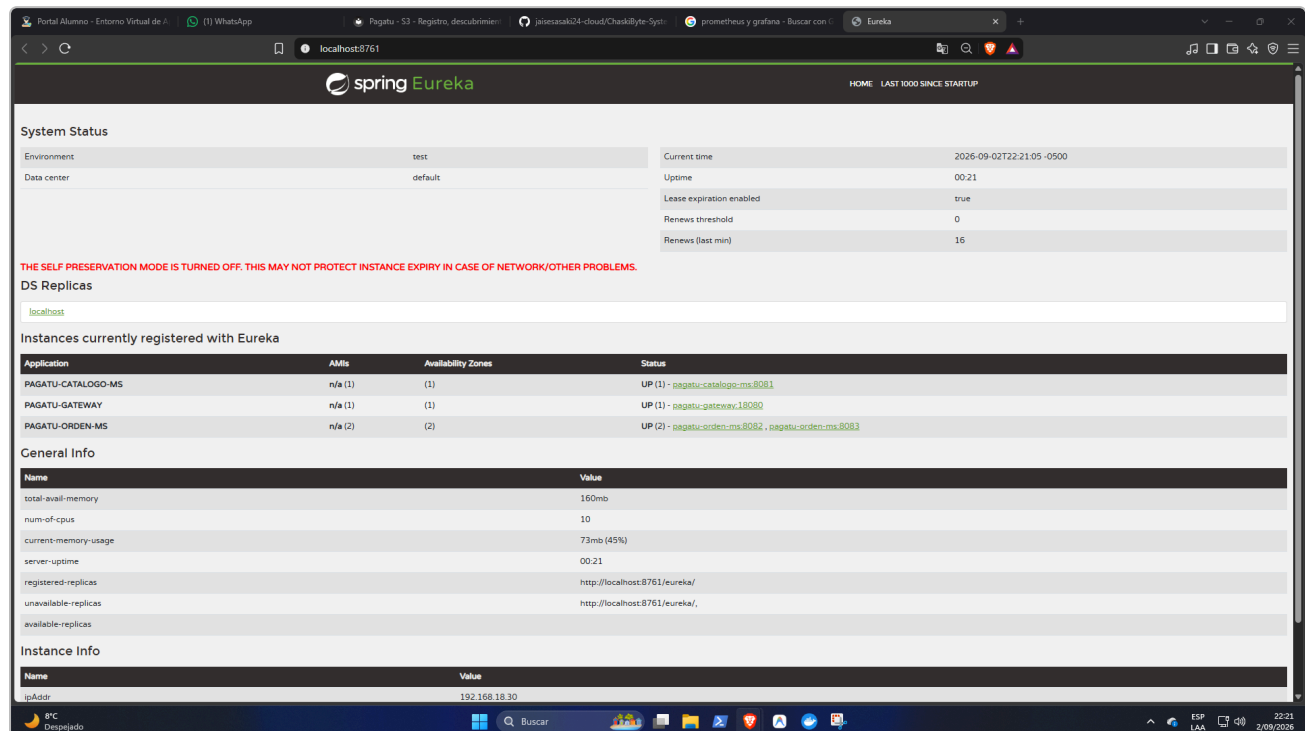


Figura 1: Service Registry Eureka (<http://localhost:8761>) mostrando a PAGATU-GATEWAY (puerto 18080), PAGATU-CATALOGO-MS (8081) y 2 instancias de PAGATU-ORDEN-MS (8082 y 8083) en estado UP con el reloj y usuario de Windows visible.

3.2 Pruebas de Persistencia Relacional y Consumo por PowerShell

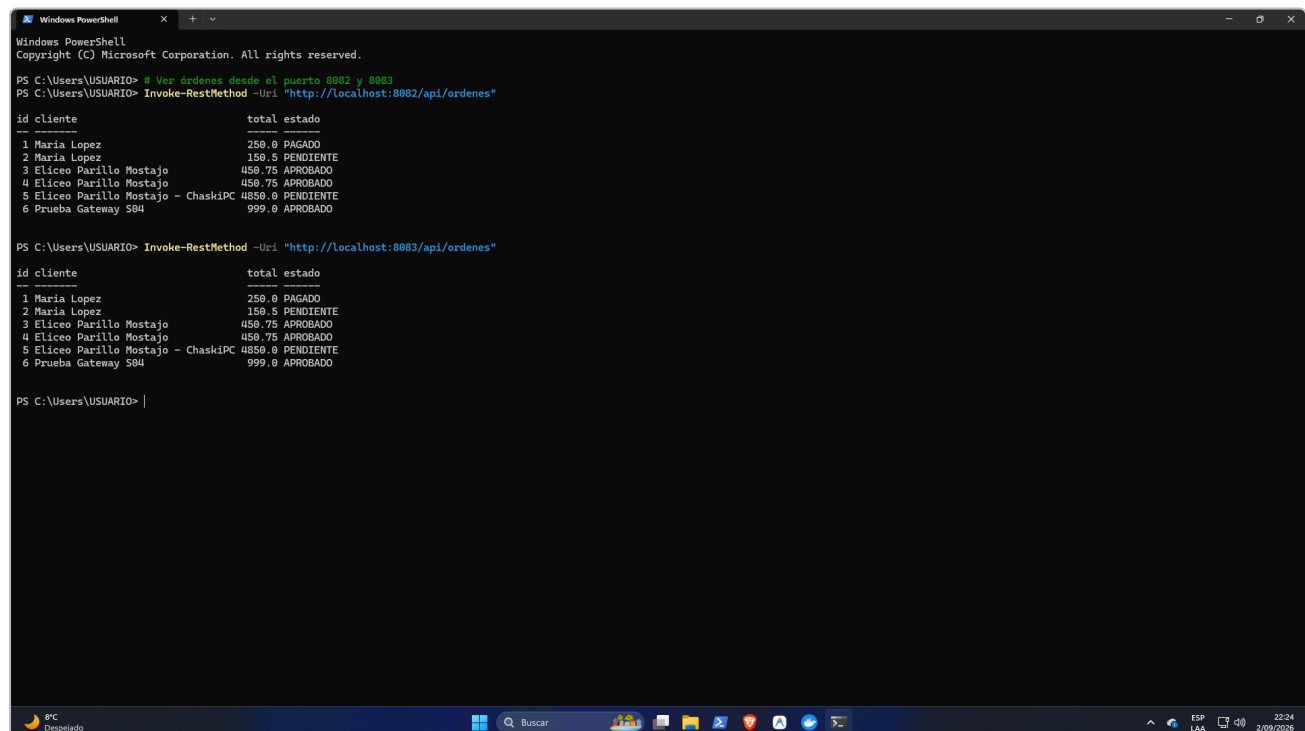


Figura 2: Verificación de órdenes en PostgreSQL (puerto 5433) respondiendo idénticamente desde el puerto 8082 y 8083, demostrando persistencia compartida.

4. Rúbrica de Evaluación de la Unidad I (Ponderación 100% - Escala Vigesimal)

Criterio de Evaluación	Nivel Obtenido	Justificación Técnica
1. Servicio REST persistente	A (20 pts)	Implementación de CRUD completo en catálogo y órdenes con manejo de códigos HTTP (200, 201, 400, 404) sobre PostgreSQL.
2. Configuración centralizada	A (20 pts)	Config Server suministrando configuraciones separadas para DEV y PROD desde config-repo sin hardcodeo en los microservicios.
3. Registro y descubrimiento	A (20 pts)	Eureka Server administrando el registro dinámico y monitorización por latidos (heartbeats) de todos los nodos.
4. Acceso por API Gateway	A (20 pts)	Spring Cloud Gateway unificando el acceso público en el puerto 18080 con enrutamiento reactivo por esquemas lb://.
5. Ejecución concurrente y balanceo	A (20 pts)	Dos réplicas activas de órdenes atendiendo peticiones distribuidas equitativamente mediante Round Robin y tolerancia a fallos.
6. Dominio propio del equipo	A (20 pts)	ChaskiPC Hardware E-Commerce estructurado con Brief Técnico completo y repositorio GitHub con los 7 topics configurados.

Nota Final de Unidad 1: 20 / 20